





DESKRIPSI

Merupakan sebuah game 2D cozy, yang tujuan utama dari game ini adalah menanam tanaman yang diberikan pada saat pertama kali main. Tantangan yang ada adalah player harus menyiram tanaman dalam waktu tertentu, dimana kalau telat menyiram maka tanaman akan layu dan hilang. Kalau berhasil maka hasil tanaman dapat dijual untuk mendapatkan coin.

Dalam world juga terdapat berbagai chest yang dapat dibuka untuk mendapatkan coin secara cepat, dan key rahasia. Player juga diwajibkan untuk mendapatkan coin sebanyak-banyaknya karena dalam permainannya salah satu untuk mendapatkan chest adalah memperbaiki jembatan yang rusak dan rumah yang rusak.

Ending game akan diberikan kepada pemain yang memperbaiki banyak infrastruktur





FEATURE GAME

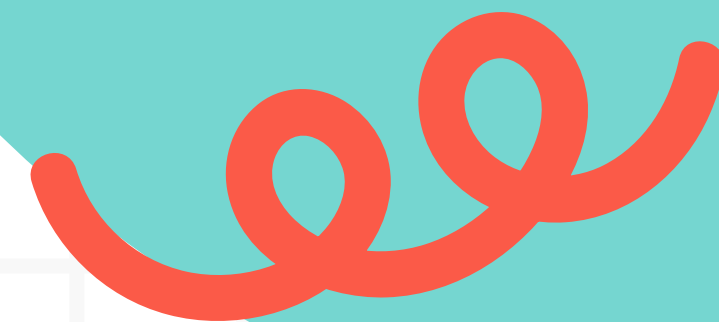
Player bisa melakukan

- Menanam seed yang diberikan
- Menyiram dan harvest tanaman
- Consume hasil panen
- Trading item
- Membuka chest
- Memperbaiki jembatan dan rumah yang rusak
- Sleep
- Kelelahan





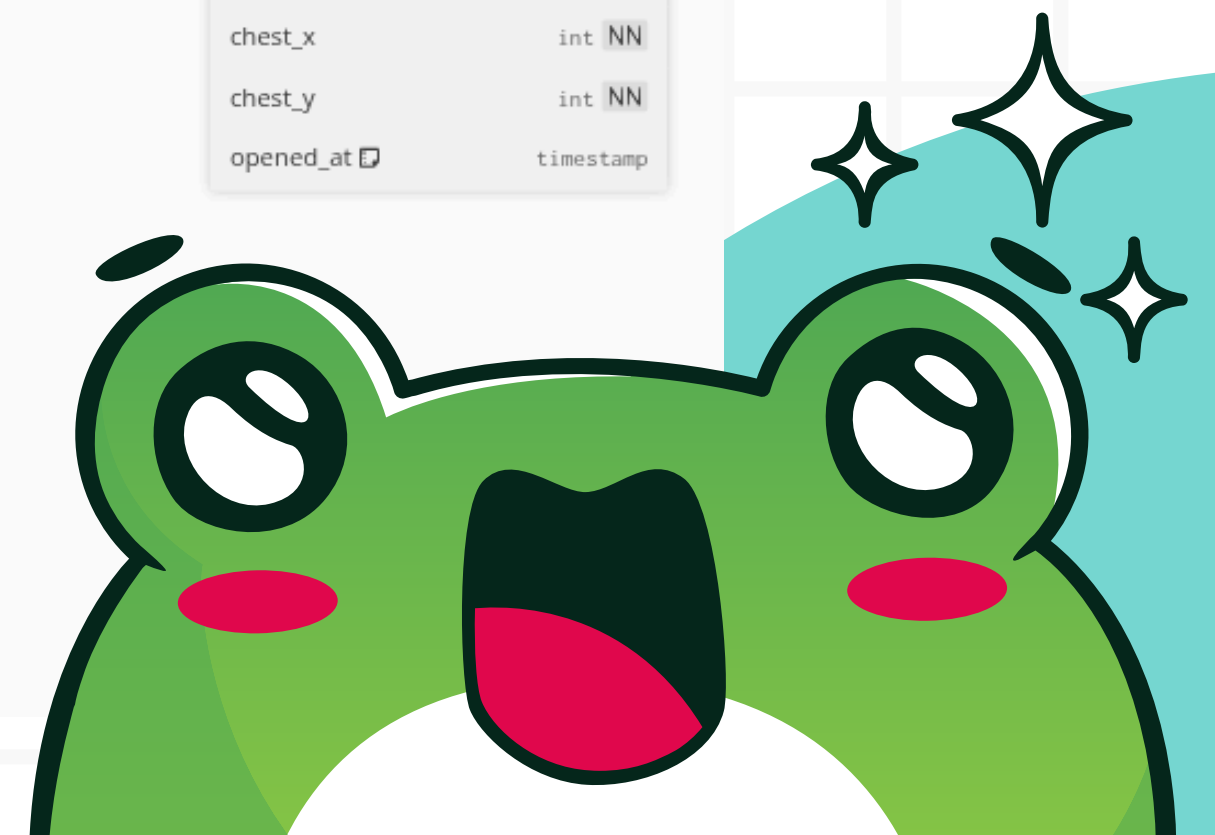
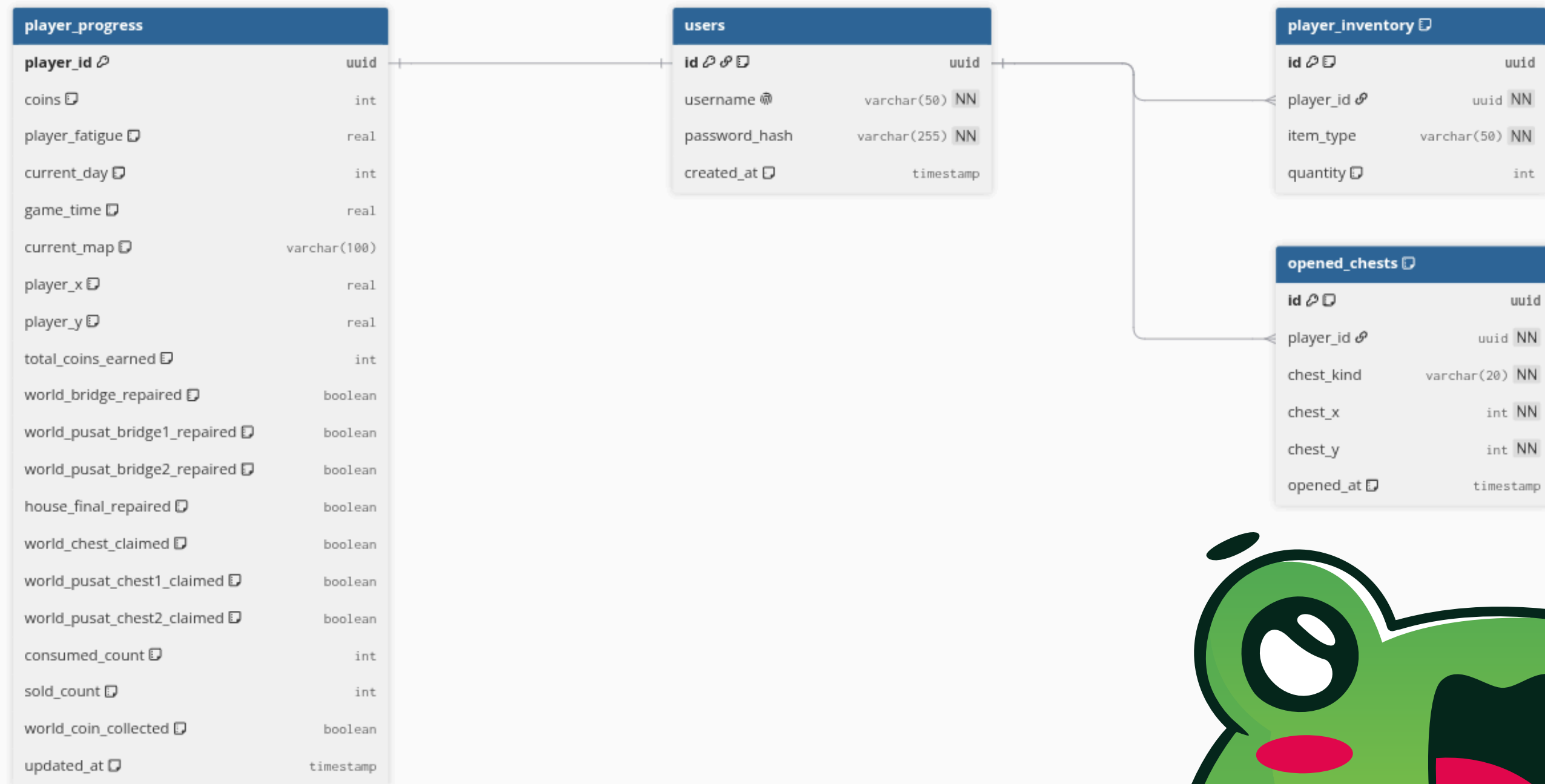
YEAY



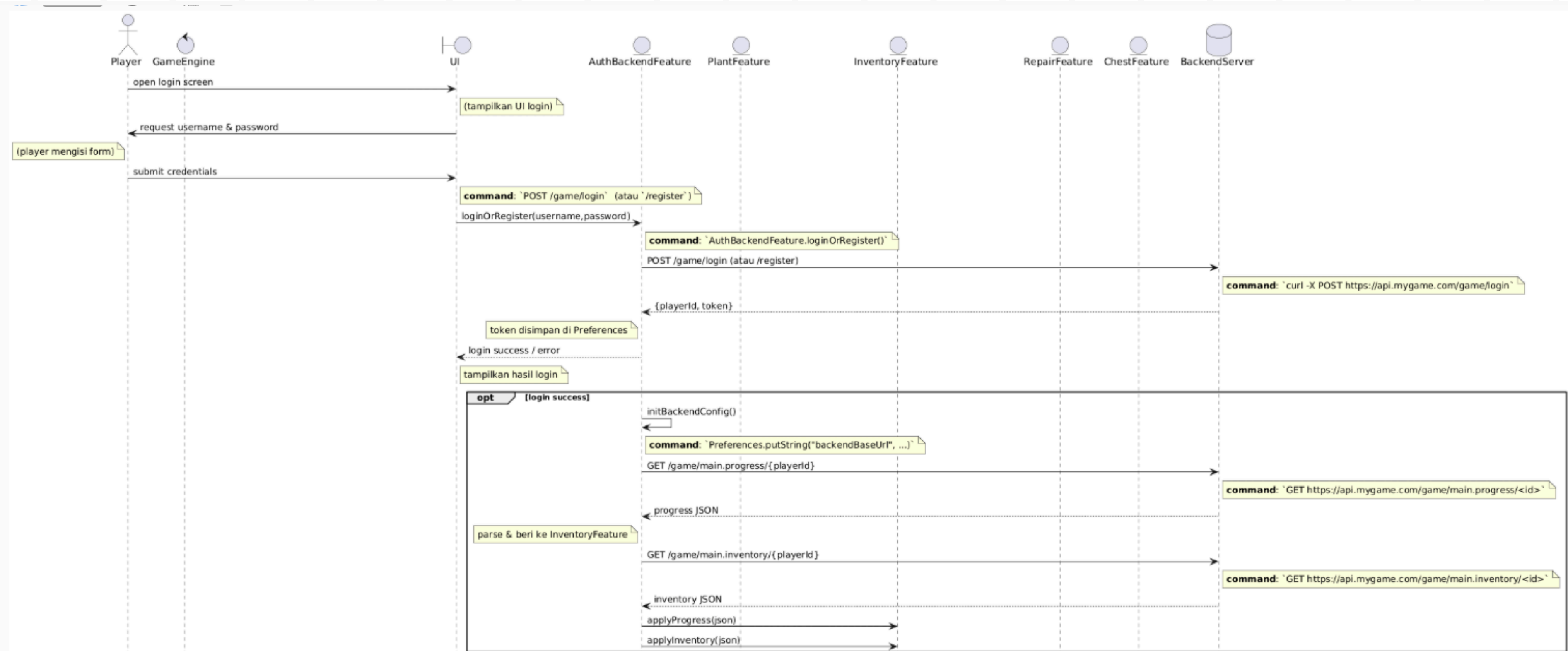
DIAGRAM



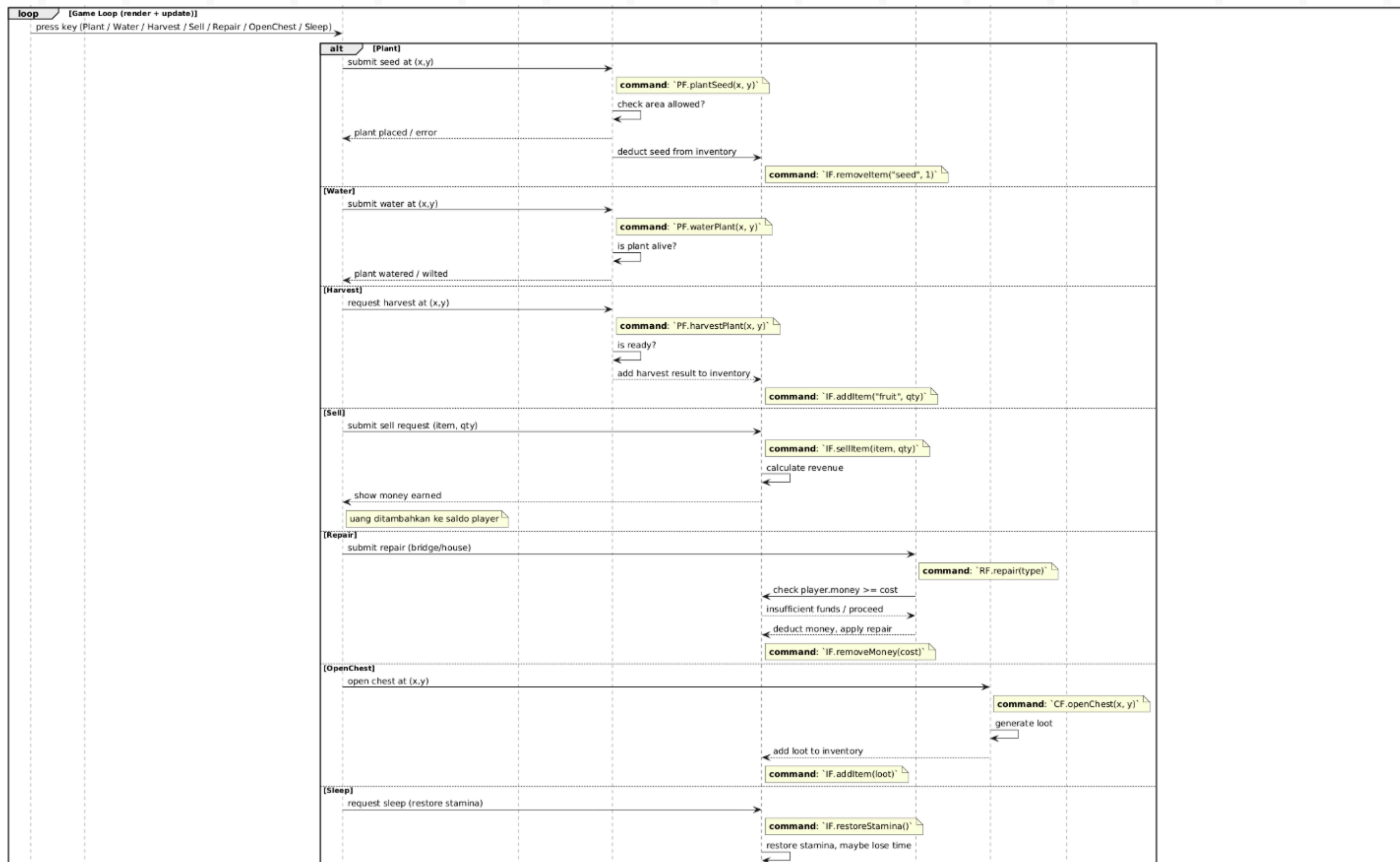
ENTITY RELATIONSHIP DIAGRAM



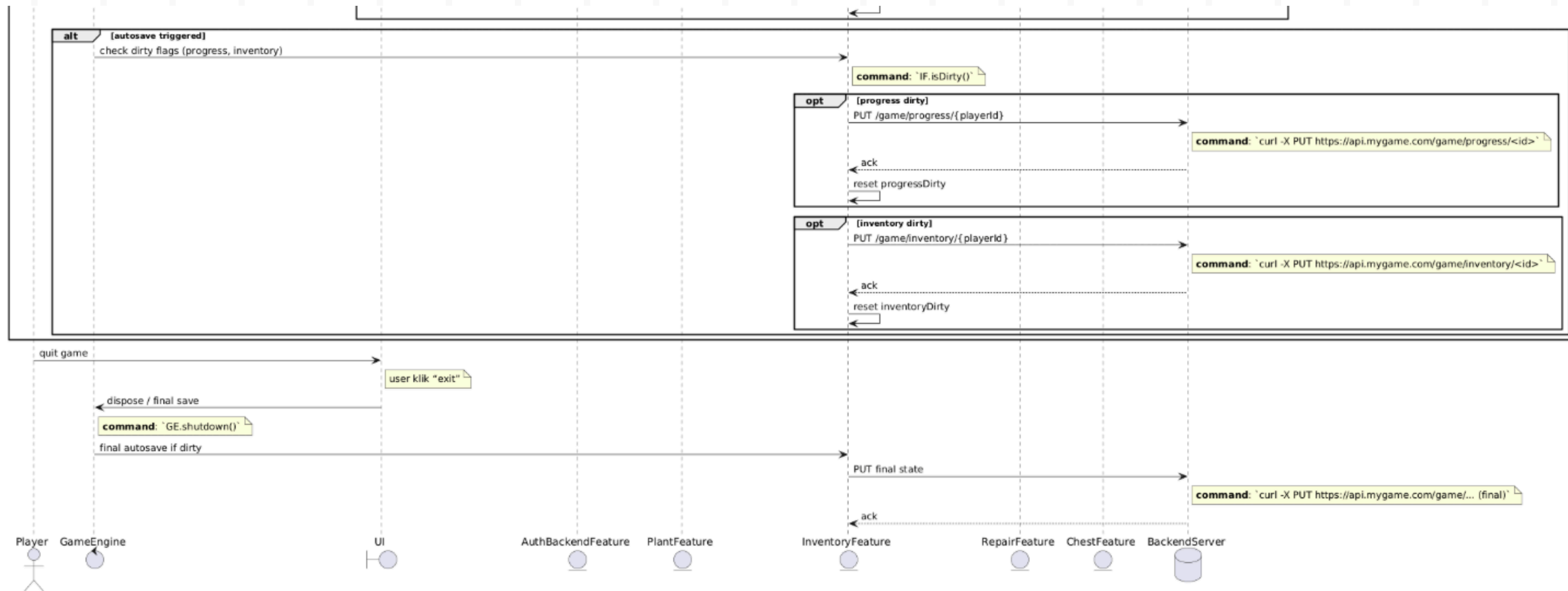
SEQUENCE DIAGRAM

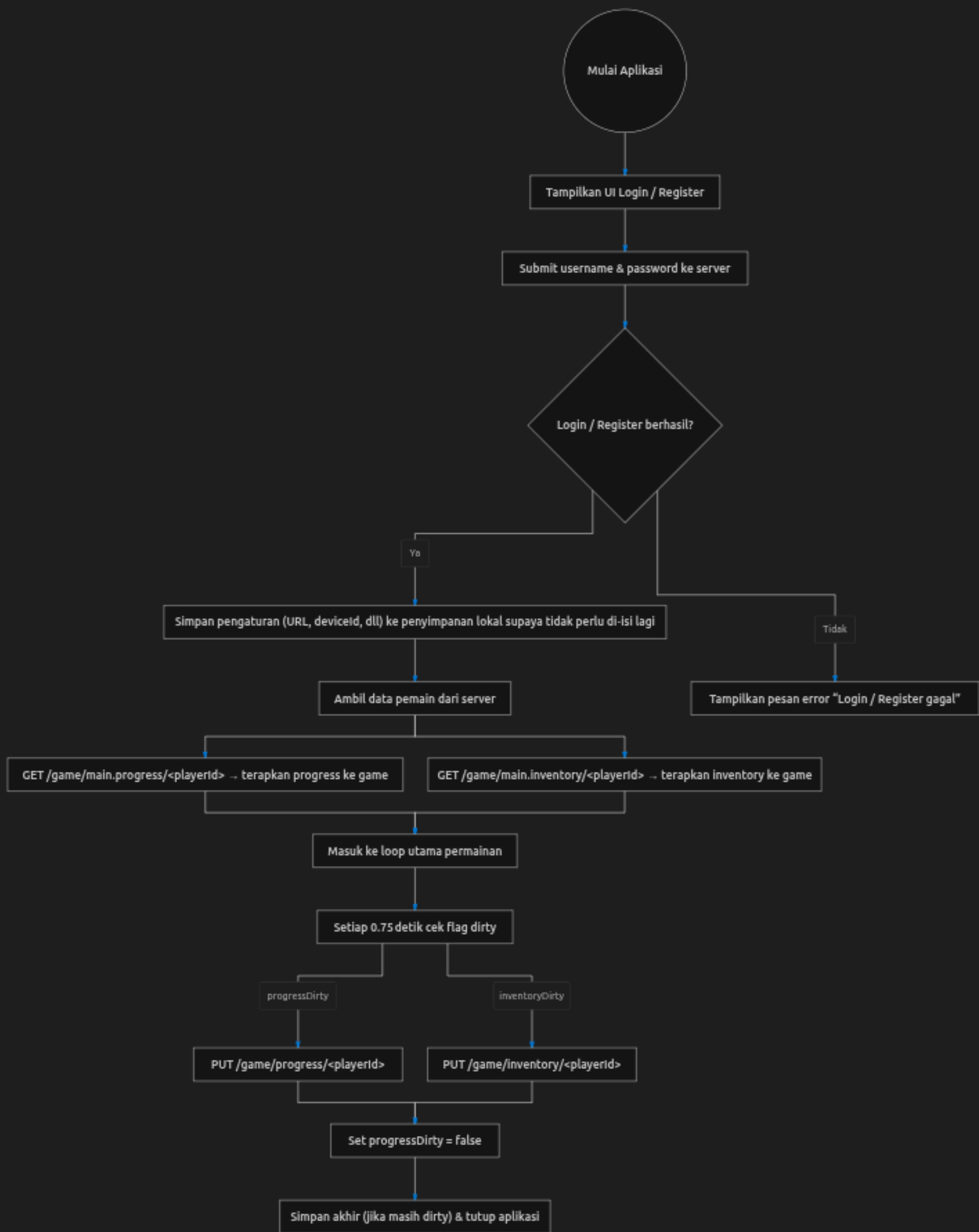


SEQUENCE DIAGRAM



SEQUENCE DIAGRAM





FLOWCHART





MIAW



DESIGN PATTER





BUILDER

Pattern Builder dipakai untuk menyusun data JSON sedikit demi sedikit sebelum dikirim ke backend. Sehingga, kita tidak perlu menulis JSON secara manual yang dimana bisa saja terjadi human error. Dengan menggunakan pattern ini, memungkinkan penulisan yang lebih rapi dan jelas, misalnya dengan pemanggilan sekaligus seperti

`'.field("coins", 500).field("current_day", 3).build()'`, sehingga proses pembuatan payload menjadi lebih mudah dipahami dan mengurangi risiko kesalahan.





COMMAND

Pattern Command digunakan untuk memisahkan feature repaired bridge dan final house ke dalam objek khusus. Jadi, saat pemain menekan button repaired, sistem tidak langsung menjalankan prosesnya, tetapi membuat objek RepairBridgeCommand lalu menyerahkannya ke Invoker untuk dieksekusi. Sehingga dengan cara ini code menjadi muda dalam debugging, dan lebih fleksibel karena aksi dapat diubah atau bahkan dikembangkan menjadi fitur undo/redo tanpa perlu mengubah kode utama yang memanggilnya.



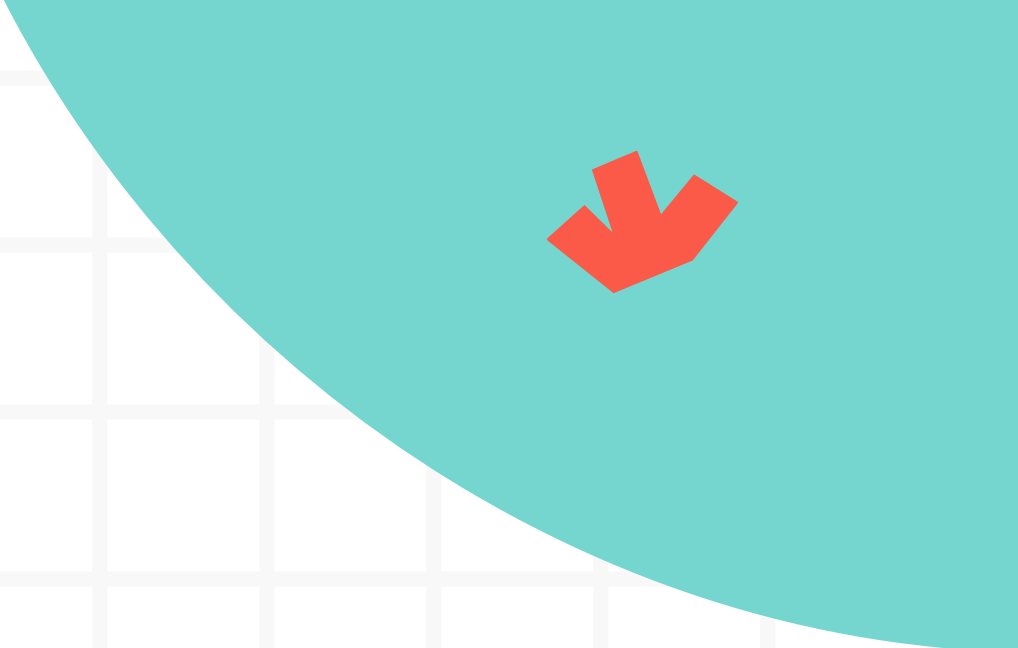


FACADE



Pattern Facade digunakan untuk menyederhanakan proses komunikasi antara game dengan backend. Sehingga, kode game cukup memanggil `backendFacade.sendProgress(url, payload)` tanpa perlu menangani detail teknis seperti koneksi HTTP, header, atau format request. Semua detail tersebut disembunyikan.

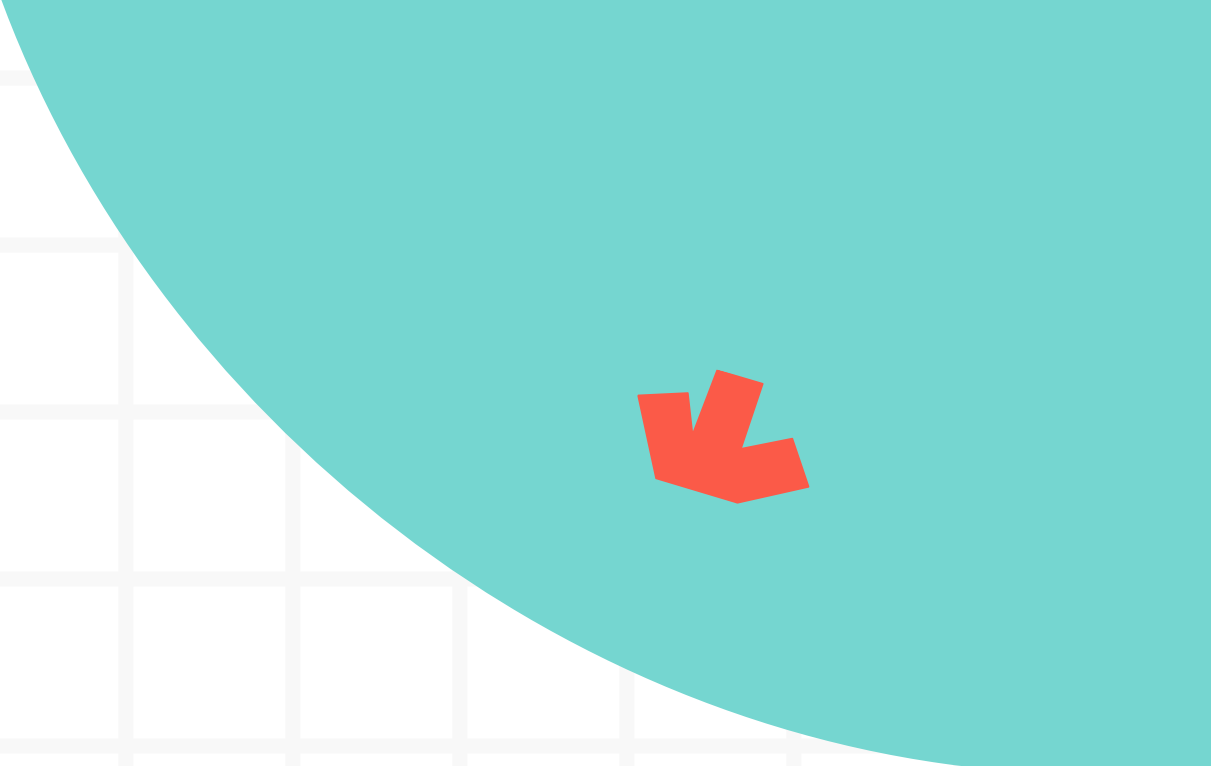




FACTORY

Pattern Factory digunakan untuk membuat komponen UI seperti Label secara terpusat dan konsisten. Sehingga, code tidak perlu berulang kali menulis `new Label(text, skin)`, tetapi cukup memanggil `uiFactory.createLabel(skin, text)`. Selain lebih rapih, pattern ini juga memudahkan nantinya ketika ada tampilan UI yang berubah secara global.





OBSERVER



Pattern Observer digunakan untuk menangani event saat game completed. GameEventManager menyimpan daftar listener, lalu ketika state completenya tercapai, maka sistem akan memanggil notifyGameCompleted() ke semua listener yang terdaftar, misalnya listener yang menampilkan layar game completed. Sehingga, logic state completed dapat dipisahkan dari logic UI, jadinya code tidak saling bergantung sama lain



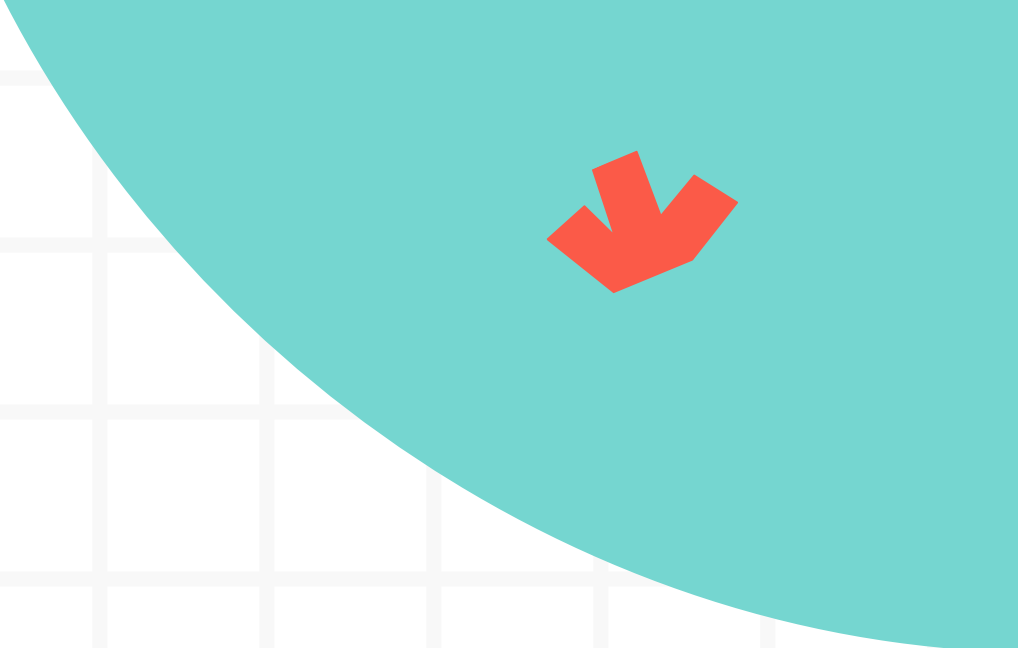


SINGLETON



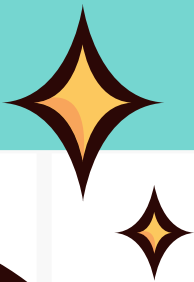
Pattern Singleton digunakan untuk memastikan hanya ada satu objek konfigurasi game yang dipakai di seluruh aplikasi. Class ini menyimpan pengaturan global seperti ukuran UI 640×480 dan diakses melalui `GameConfig.get()`. Dengan begitu, semua bagian program menggunakan data konfigurasi yang sama, sehingga ukuran dan tampilan UI tetap konsisten tanpa risiko adanya konfigurasi yang berbeda-beda.





STRATEGY

Pattern Strategy digunakan untuk memisahkan bagaimana karakter bergerak. Interface MovementStrategy digunakan sebagai general rulanya melalui method apply(dx, dy), sedangkan DefaultMovementStrategy merupakan implementasi yang digunakan untuk menjalankan gerakan menggunakan game.moveBy(). Dengan pendekatan ini, sistem menjadi lebih fleksibel karena jenis gerakan baru, seperti berenang, berlari, atau efek status tertentu, dapat ditambahkan hanya dengan membuat class strategi baru tanpa perlu mengubah code utama game.

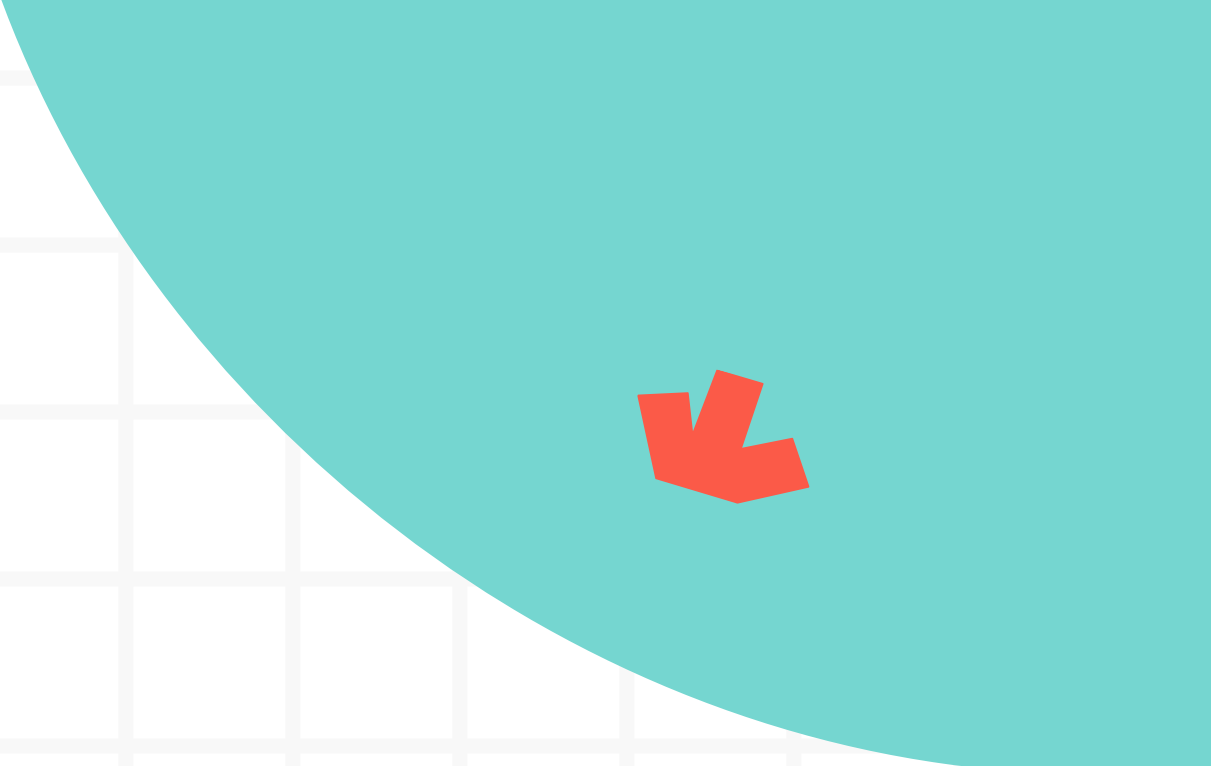




MIAW

BACKEND

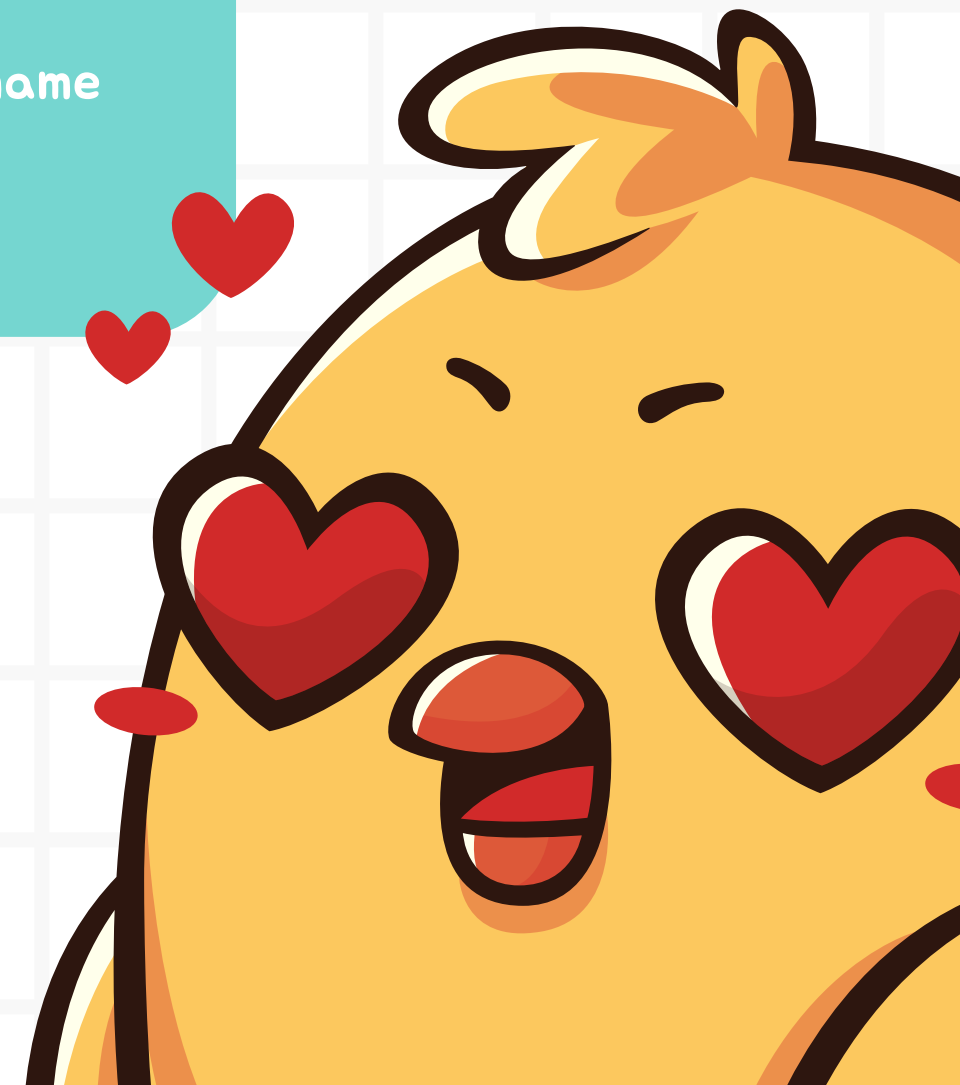




/REGISTER (POST)



Digunakan untuk mendapatkan data terkait user yang mau register yaitu username dan passwod. Lalu membuat account baru untuk user tersebut.

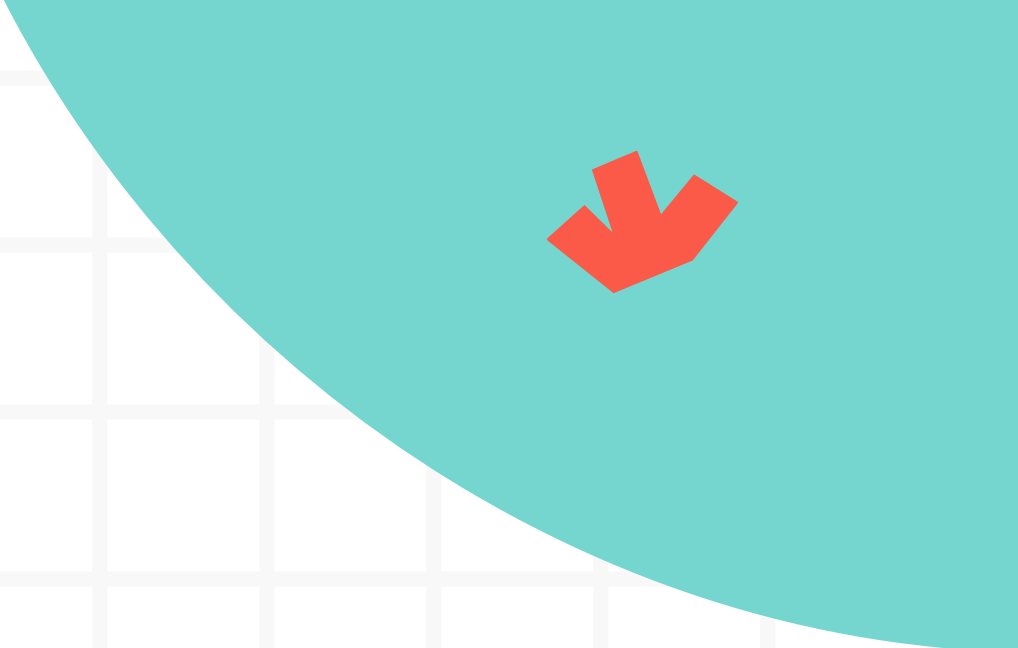




/LOGIN (POST)

Digunakan untuk melakukan validasi terkait username dan password player, serta akan mereturn token auth yang akan digunakan untuk auth nantinya

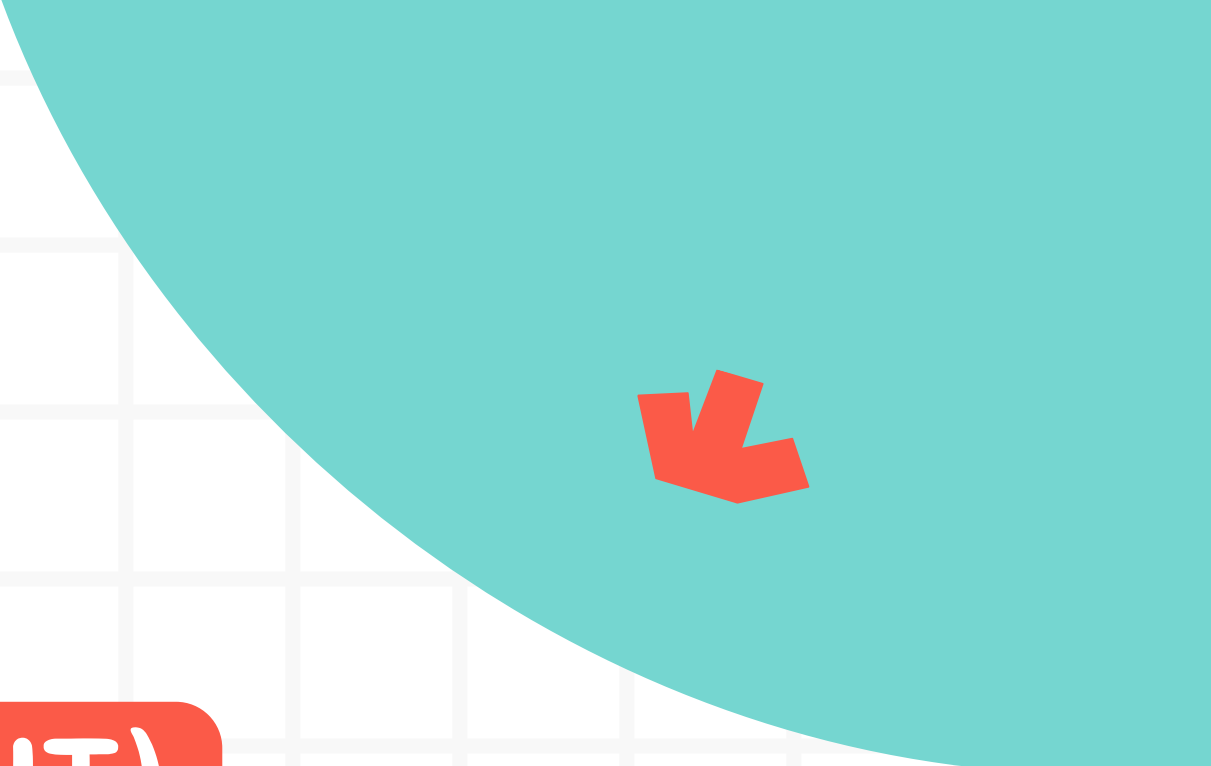




/PROGRESS/:PLAYERID (GET)

Digunakan untuk mendapatkan seluruh status terkait progress dari player seperti level, score, dan lainnya dengan id tertentu.

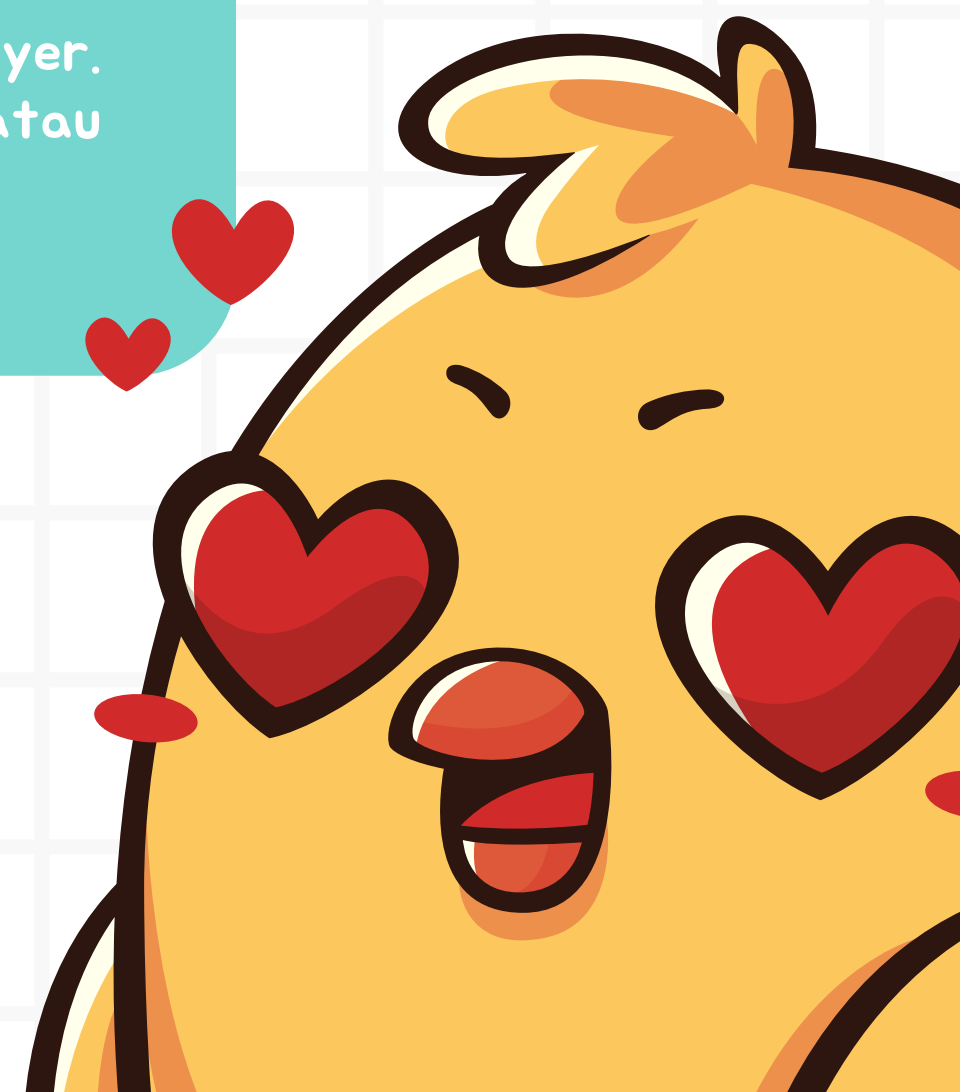




/PROGRESS/:PLAYERID (PUT)



Digunakan untuk menyimpan atau memperbarui status permainan dari player. Misalkan ketika player sudah mendapatkan chest, atau memperbaiki bridge atau house maka progress-progress tersebut akan disimpan.

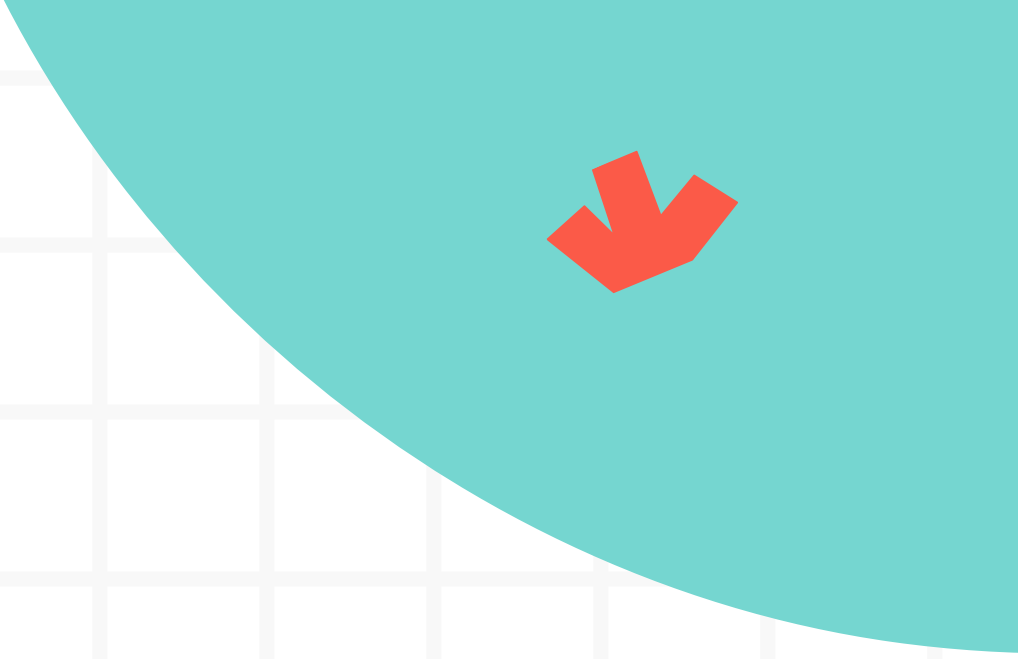




/INVENTORY/:PLAYERID (GET)

Digunakan untuk mendapatkan berbagai item yang dimiliki oleh player.

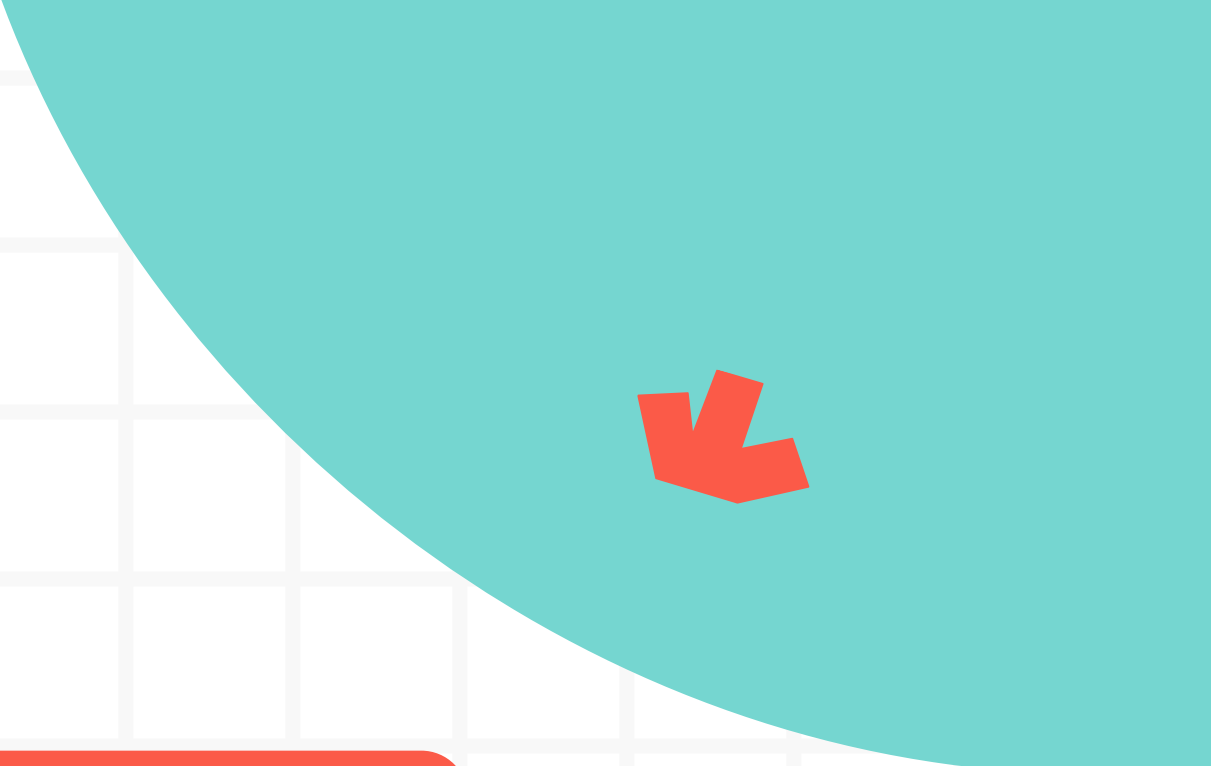




`/INVENTORY/:PLAYERID (PUT)`

Digunakan untuk menyimpan seluruh item yang ada di inventory player ketika terjadi perubahan secara langsung.

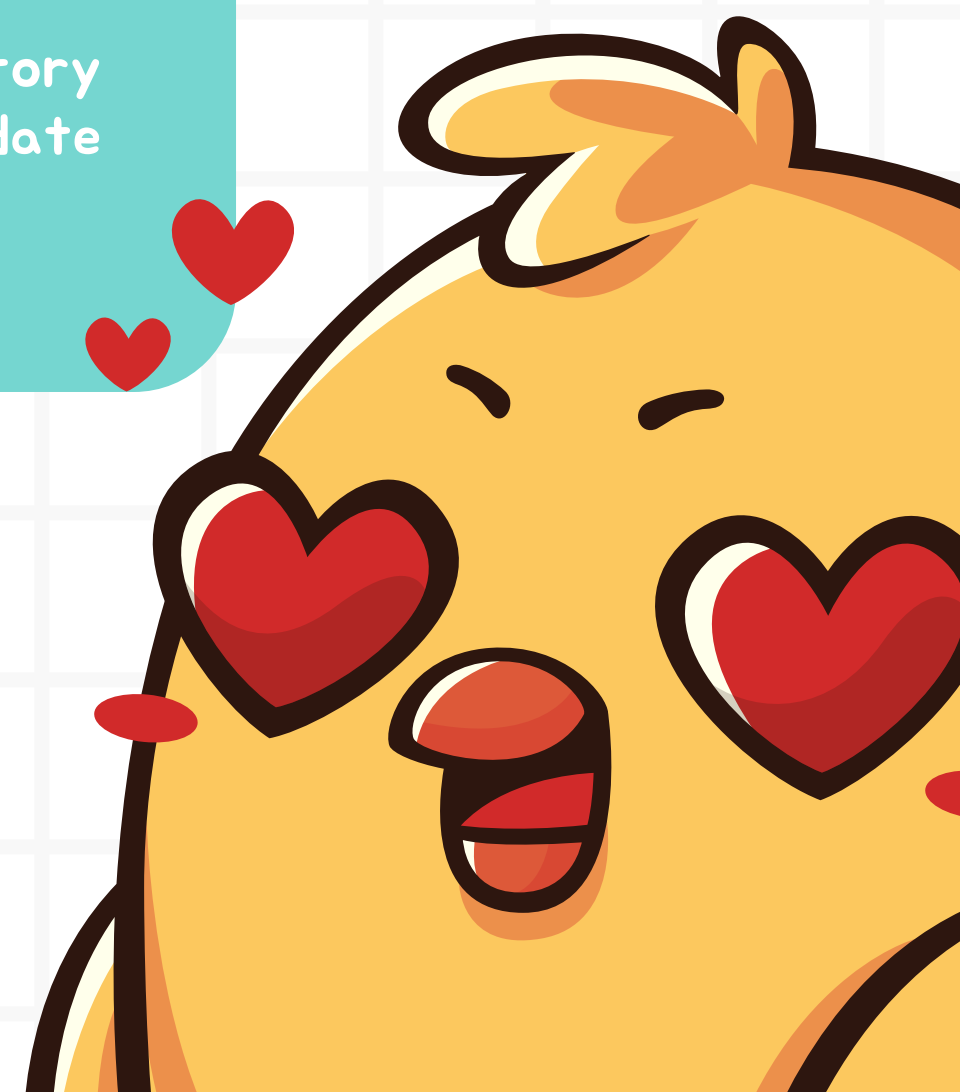




/INVENTORY/:PLAYERID/ITEM (POST)



Digunakan untuk menambahkan satu item baru yang akan ditambahkan ke inventory player, bedanya dengan route sebelumnya adalah route ini tidak mengupdate inventory secara keseluruhan.



**THANK YOU
FOR PLAYING**

